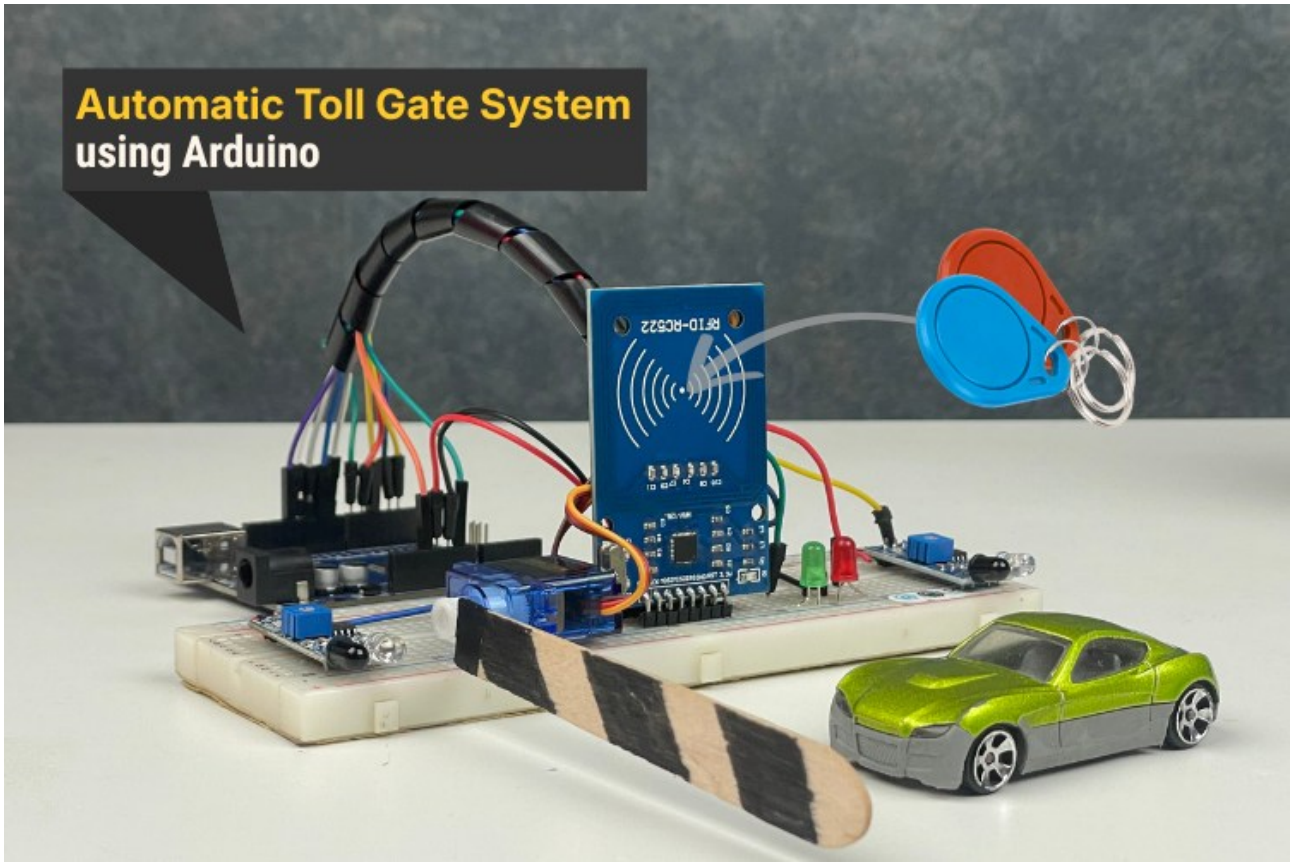
Smart Automatic Toll Gate System Using Arduino & RFID

Create an automated toll gate with Arduino that detects vehicles, verifies RFID payment cards, deducts tolls, and controls a servo-driven gate seamlessly.



Components and supplies

- 1 Ring with 12 RGB WS2812 LEDs and integrated driver
- 1 RS422 / RS485 Shield for Arduino UNO
- 1 Servo Motor SG90 180 degree
- 1 RFID Reader Module
- 1 ir sensor

Project Overview

This is a Smart $ Automatic Toll Gate System Project Using Arduino $, RFID cards, and basic sensor modules to automate toll collection. Designed for learning and real-world relevance, this system detects vehicles, authenticates payments via RFID, opens a servo-controlled toll barrier, and resets for the next vehicle — all without manual intervention.

This toll gate automation demonstrates how embedded systems and low-cost components can transform traditional toll booths into contactless, efficient, and scalable solutions — ideal for student projects, IoT demos, and smart city prototypes.

How It Works

The automated toll system operates through a clear, event-driven workflow:

1. Vehicle Detection

   An IR sensor at the entrance detects an approaching vehicle and triggers the system to start listening for an RFID card.

2. RFID Authentication

   Drivers tap their RFID card near the RC522 reader. The Arduino reads its unique ID and checks if it’s stored in the onboard database with sufficient balance.

3. Validation Logic

   Valid card + sufficient balance: Toll is deducted automatically.

   Invalid card / no balance: Access is denied; red LED lights up.

   Unregistered card: Rejected with error feedback.

4. Gate Control

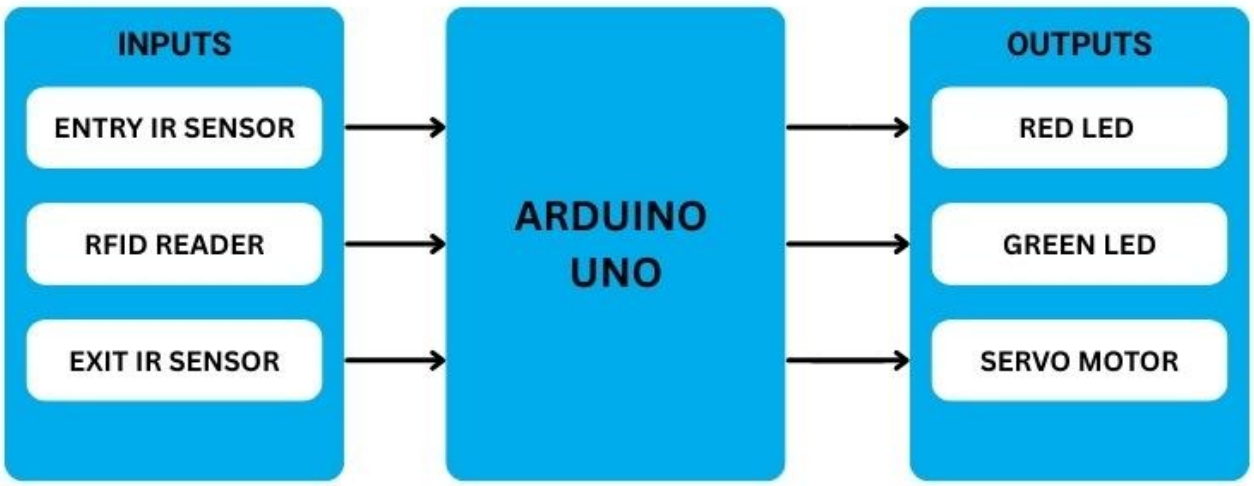
   Upon successful payment:

   Green LED turns on.

   The servo motor rotates to open the gate (~90°).

   On exit detection by the second IR sensor, the gate closes automatically.

   The flow repeats for subsequent vehicles without resetting the Arduino manually.



Wiring & Connections

Below is a concise reference for wiring major components:

RFID RC522 Reader

RFID Pin	Arduino
SDA	D10
SCK	D13
MOSI	D11
MISO	D12
RST	D9
3.3V	3.3V
GND	GND

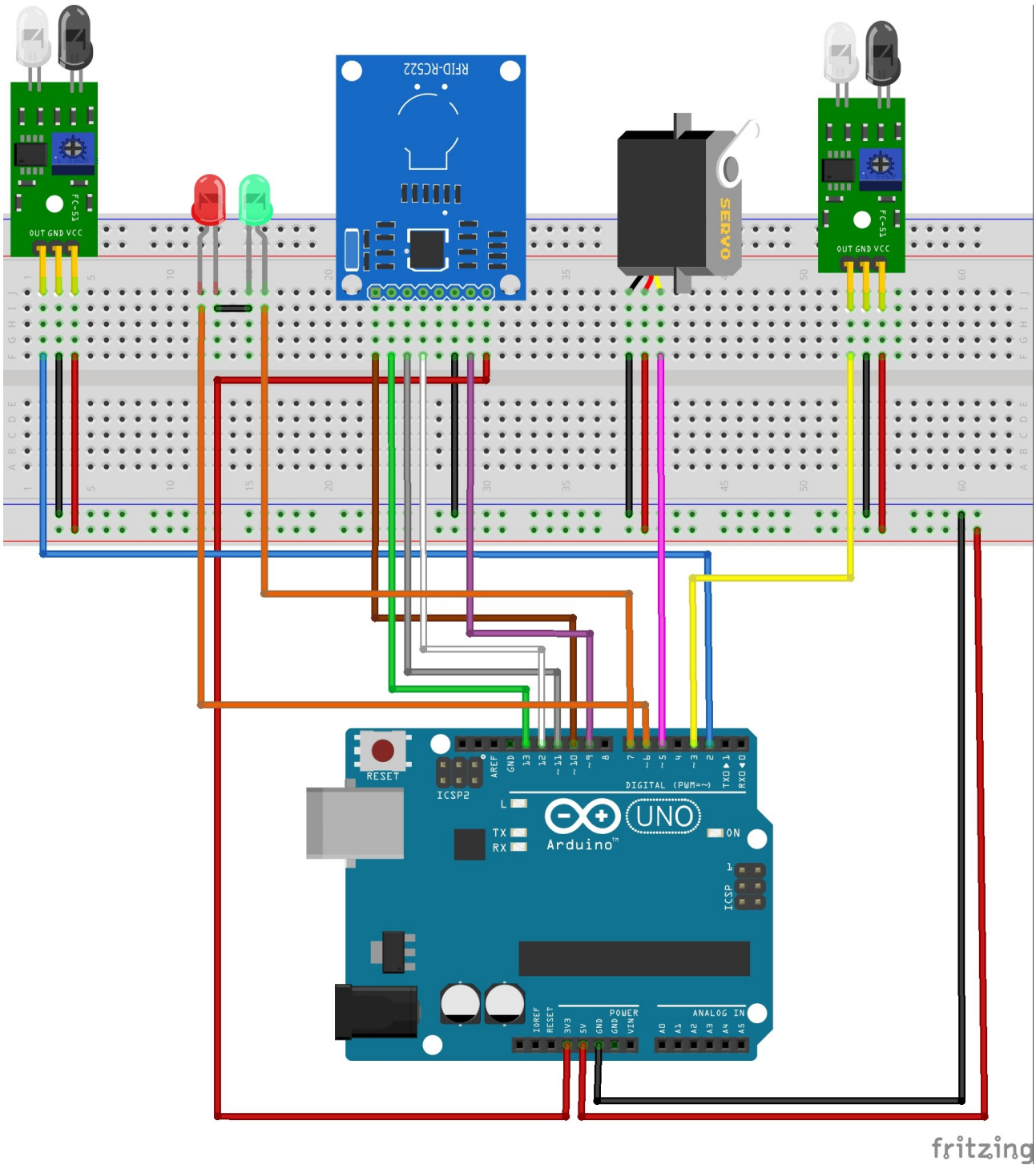
IR Sensors

IR Sensor	Arduino
Entry Sensor	OUT D2
Exit Sensor	OUT D3
VCC	5V
GND	GND
LED	Indicators
LED	Arduino
Red	D6
Green	D7
GND	GND

Servo Motor

Servo Pin	Arduino
Signal	D5
VCC	5V
GND	GND

All sensors share a common ground. The RFID uses 3.3V logic, so confirm proper voltage to prevent damage.



Arduino Code Highlights

Your Arduino sketch includes:

- Libraries: SPI (communication), MFRC522 (RFID), Servo (gate control)
- Pin Definitions & Setup: Map hardware pins to intuitive constants

- Main Loop: Listen for vehicle presence (IR) and RFID card
- Authenticator: Matches UID against allowed cards and checks balance
- Servo Control: Opens/closes the gate and updates LEDs accordingly onde ele está. Basta usar a opção Personalizar Sumário para fazer esse tipo de alteração e o Word fará a alteração.

```
#include <SPI.h>
#include <MFRC522.h>
#include <Servo.h>
#define SS_PIN 10
#define RST_PIN 9
#define IR_ENTRY 2
#define IR_EXIT 3
#define LED_RED 6
#define LED_GREEN 7
#define SERVO_PIN 5
Servo gateServo;
MFRC522 mfrc522(SS_PIN, RST_PIN);
void setup() {
   Serial.begin(9600);
   SPI.begin();
   mfrc522.PCD_Init();
   gateServo.attach(SERVO_PIN);
   pinMode(IR_ENTRY, INPUT);
   pinMode(IR_EXIT, INPUT);
   pinMode(LED_RED, OUTPUT);
   pinMode(LED_GREEN, OUTPUT);
   gateServo.write(0); // Start with gate closed
}
```

More logic handles UID comparison, balance update, and event sequencing.

Real-World Applications

Beyond a classroom project, this architecture has practical relevance:

1. **Highway Toll Plazas:** Faster and contactless tolling.

2. **Parking Management:** Automated entry and exit tracking.
3. **Gated Communities:** Efficient resident vehicle access.
4. **Industrial Facilities:** Track truck movement and permit access.

Future Enhancements

Once the base system is working, you can expand it with:

1. **LCD Display:** Show balances, messages, and system status.
2. **Database Logging:** Store transaction history on SD or cloud (ESP32/Firebase).
3. **Mobile App Link:** View and manage balances remotely.
4. **OCR License Plate Recognition:** Link vehicles to RFID identity.
5. **Solar Power Integration:** Make toll booths energy independent.

Downloadable files

https://github.com/VedhathiriK/Built-Your-Own-GPS-Tracker-with-Geofence-Using-Xiao-ESP32-S3-/archive/refs/heads/main.zip

References

https://projecthub.arduino.cc/rinme/smart-automatic-toll-gate-system-using-arduino-rfid-105b72